

SMART MOTION AND ZONE – BASED FAN CONTROL SYSTEM

MINI PROJECT REPORT

Submitted by

CHRISTINA DIXON (ASI22CA022)

MALAVIKA V R (ASI22CA041)

MERIN PHILIP (ASI22CA044)

NANDANA E S (ASI22CA047)

Under the guidance of

Asst. Prof. SABITHA M G

To

APJ Abdul Kalam Technological University
in partial fulfillment of the requirements for the award
of the Degree of
*Bachelor of Technology in Computer Science and
Engineering (AI)*

Department of Artificial Intelligence and Data Science



S6 CSE(AI)

2022 – 2026

ADI SHANKARA INSTITUTE OF ENGINEERING AND TECHNOLOGY

Kalady- 683574

April 2025

SMART MOTION AND ZONE – BASED FAN CONTROL SYSTEM

MINI PROJECT REPORT

Submitted by

CHRISTINA DIXON (ASI22CA022)

MALAVIKA V R (ASI22CA041)

MERIN PHILIP(ASI22CA044)

NANDANA E S (ASI22CA047)

Under the guidance of

Asst. Prof. SABITHA M G

To

APJ Abdul Kalam Technological University
in partial fulfillment of the requirements for the
award of the Degree of
*Bachelor of Technology in Computer Science and
Engineering (AI)*

Department of Artificial Intelligence and Data Science



S6CSE(AI)

2022 – 2026

**ADI SHANKARA INSTITUTE OF ENGINEERING AND
TECHNOLOGY**

Kalady- 683574

April 2025

**ADI SHANKARA INSTITUTE OF ENGINEERING AND
TECHNOLOGY**
Kalady- 683574



Department of Artificial Intelligence and Data Science

CERTIFICATE

Certified that this is a bonafide record of the project entitled

“Smart Motion and Zone – Based Fan Control System”

Submitted by

CHRISTINA DIXON (ASI22CA022)

MALAVIKA V R (ASI22CA041)

MERIN PHILIP(ASI22CA044)

NANDANA E S (ASI22CA047)

*during the year 2024-25 in partial fulfillment of the
requirement for the award of the degree of
Bachelor of Technology in Computer Science and
Engineering (AI)*

Project Guide

Project Coordinator

Head of the Department

**ADI SHANKARA INSTITUTE OF ENGINEERING AND
TECHNOLOGY
Kalady- 683574**



Department of Artificial Intelligence and Data Science

CERTIFICATE

This is to certify that the project entitled “**SMART MOTION AND ZONE – BASED FAN CONTROL SYSTEM**” has been successfully carried out by **CHRISTINA DIXON (ASI22CA022), MALAVIKA V R (ASI22CA041), MERIN PHILIP (ASI22CA044), NANDANA E S (ASI22CA047)** in partial fulfillment of the Course Bachelor of Technology.

Date:

HEAD OF DEPARTMENT

DECLARATION

I/We, **CHRISTINA DIXON (ASI22CA022), MALAVIKA V R (ASI22CA041), MERIN PHILIP (ASI22CA044), NANDANA E S (ASI22CA047)**, hereby declare that the project work entitled “**SMART MOTION AND ZONE – BASED FAN CONTROL SYSTEM**”, submitted for partial fulfillment of the requirements for the award of degree of Bachelor of Technology of APJ Abdul Kalam Technological University, Kerala is a bonafide work done by us under supervision of **ASSISTANT PROFESSOR SABITHA M G**. This submission represents our ideas in our own words and where ideas or words of others have been included, we have adequately and accurately cited and referenced the original sources. I/We also declare that we have adhered to ethics of academic honesty and integrity and have not misrepresented or fabricated any data or idea or fact or source in our submission. I/We understand that any violation of the above will be a cause for disciplinary action by the institute and/or the University and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been obtained.

PLACE: KALADY

DATE:

SIGNATURE

CHRISTINA DIXON

MALAVIKA V R

MERIN PHILIP

NANDANA E S

ACKNOWLEDGMENT

In the name of almighty **GOD**, I express my sincere thanks to him keeping me fit for successful completion of the project.

I am thankful to **Dr. M S MURALI, Principal, ADI SHANKARA INSTITUTE OF ENGINEERING AND TECHNOLOGY** for his kind support in all respect during our study.

I sincerely extend our thanks to our **CTO, Prof. P V RAJARAMAN**, for all the help, motivation and guidance to carry out this project.

I sincerely thank **Dr. SARIKA S, HOD, Department of Artificial Intelligence and Data Science, ADI SHANKARA INSTITUTE OF ENGINEERING AND TECHNOLOGY**, for her encouragement to carry out this project.

I especially thank my Guide **Asst. Prof. SABITHA M B**, Department of Artificial Intelligence and Data Science, **ADI SHANKARA INSTITUTE OF ENGINEERING AND TECHNOLOGY**, in examining the draft of this seminar and for valuable suggestions.

I want to thank the Department of Artificial Intelligence and Data Science for giving me the permission to prepare the project on the topic “**SMART MOTION AND ZONE – BASED FAN CONTROL SYSTEM**”.

CHRISTINA DIXON

MALAVIKA V R

MERIN PHILIP

NANDANA E S

ABSTRACT

This project presents a **SMART MOTION AND ZONE-BASED FAN CONTROL SYSTEM** that leverages YOLO (You Only Look Once) object detection to provide intelligent and efficient fan control in real-time. The system is designed to detect human motion and presence in predefined zones of a room and according to that turn ON/OFF the fan. It also incorporates a web-based interface for monitoring and manual control, making the solution both practical and user-friendly.

The YOLO model is used to process live video feeds and accurately detect individuals in the room, mapping their locations to specific zones. A microcontroller (e.g., Arduino, ESP32, or Raspberry Pi) bridges the gap between the detection system and the fan hardware. The web interface provides users with a live video stream of the room, displays active zones, and offers manual override options to control the fan. The platform supports deployment on both local servers and cloud environments, ensuring accessibility and scalability.

This innovative system aims to enhance energy efficiency and user comfort by combining computer vision, IoT, and web technologies. It provides a cost-effective solution to modernize traditional fan control mechanisms while optimizing resource usage. Index Terms—**YOLO Object Detection, Smart Fan Control, Zone-Based Automation, IoT, Web Interface.**

CONTENTS

Chapter	Title	Page No
1	INTRODUCTION	1
	1.1 Background	2
	1.2 Existing System	2
	1.3 Problem Statement	2
	1.4 Scope	3
2	LITERATURE SURVEY	4
3	PROPOSED SYSYTEM	6
	3.1 Objective	7
	3.2 Methodology	7
	3.2.1 System Architecture	7
	3.2.2 Fundamental Requirements	8
	3.2.3 Non – Functional Requirements	9
	3.2.4 Hardware Requirements	9
	3.2.5 Software Requirements	9
	3.2.6 Life Cycle Model	9
	3.2.7 Feasibility Study	10
4	SYSTEM DESIGN	11
	4.1 Use Case Diagram	12
	4.2 Class Diagram	13
	4.3 Activity Diagram	14
	4.4 Deployment Diagram	15
	4.5 GUI Design	16
5	ARCHITECTURE	17

	5.1 Flow Chart	18
	5.2 Data Collection	19
	5.3 Module Description	19
6	TESTING	20
	6.1 Test Case of Module 1	21
	6.2 Test Case of Module 2	21
	6.3 Test Case of Module 3	21
	6.4 Test Case of Module 4	22
	6.5 Summary of Testing	22
7	SAMPLE CODE	23
	7.1 Frontend Code	24
	7.2 Backend Code	31
	7.3 Microcontroller Code	38
8	RESULT AND ANALYSIS	40
	8.1 Screenshots	41
	8.1.1 Home Page	41
	8.1.2 Zone Detection	41
	8.1.3 Fan Control (Auto Mode)	42
	8.1.4 Fan Control (Manual Mode)	42
9	CONCLUSION	43
10	FUTURE ENHANCEMENT	45
11	REFERENCES	47

LIST OF FIGURES

Figure No:	Figure Title	Page No
3.1	System Architecture	7
4.1	Use Case Diagram	12
4.2	Class Diagram	13
4.3	Activity Diagram	14
4.4	Deployment Diagram	15
4.5	GUI Diagram	16
5.1	Work Flow	18
8.1	Home Page	41
8.2	Zone Detection	42
8.3	Fan Control in Auto Mode	42
8.4	Fan Control in Manual Mode	42

LIST OF TABLES

Table No:	Table Title	Page No
2.1	Literature Survey	5
6.1	Test Case Results for Module 1	21
6.2	Test Case Results for Module 2	21
6.3	Test Case Results for Module 3	21
6.4	Test Case Results for Module 4	22
8.1	Performance Analysis	41

ABBREVIATION

AI	Artificial Intelligence
API	Application Programming Interface
CSS	Cascading Style Sheet
CTC	Connectionist Temporal Classification
CV	Computer Vision
ESP	Espressif Systems Programmable System – on - Chip
GPU	Graphics Processing Unit
GUI	Graphical User Interface
HTML	Hypertext Markup Language
HTTP	Hypertext Transfer Protocol
IDE	Integrated Development Environment
IoT	Internet of Things
ML	Machine Learning
PAF	Part Affinity Fields
RNN	Recurrent Neural Network
TC	Test Case
YOLO	You Only Look Once

Chapter 1

Introduction

1.1 Background

In modern homes and workplaces, maintaining an optimal environment for comfort and energy efficiency is crucial. Traditional fan control systems rely on manual operations, often leading to unnecessary energy consumption. To address this, automation and intelligent systems are gaining prominence.

The **Smart Motion and Zone-Based Fan Control System Using YOLO** is designed to provide real-time control of fans based on human presence detection. By leveraging **YOLO (You Only Look Once)** object detection, the system ensures that fans operate only when a person is detected in a particular zone. Additionally, the integration of **IoT** and a **web-based interface** allows seamless monitoring and control, providing both automatic and manual modes of operation.

1.2 Existing System

Existing fan control systems often rely on the following approaches:

- **Manual Operation:** Users switch fans on or off using wall-mounted controls.
- **Timer-Based Systems:** Fans operate on preset timers without considering real-time occupancy.
- **Basic Motion Sensors:** Motion-activated systems detect movement but may fail to differentiate between humans and other objects, leading to false triggers.

These systems are inefficient in terms of energy management and often lack the ability to provide real-time feedback and remote control.

1.3 Problem Statement

The primary challenges addressed by this project include:

- **Energy Waste:** Fans often remain on in unoccupied zones, resulting in excessive energy consumption.
- **Lack of Automation:** Manual control is inefficient and inconvenient.
- **Limited Customization:** Most existing systems do not offer adaptive or zone-based control.
- **No Real-Time Monitoring:** Users are unable to monitor or control the fan remotely.

1.4 Scope

The **Smart Motion and Zone-Based Fan Control System Using YOLO** focuses on the following objectives:

- **Real-Time Zone-Based Control:** YOLO detects human presence and determines which zone is occupied.
- **Energy Efficiency:** Fans are automatically switched on or off based on occupancy.
- **Manual Control:** Users can override the automated system using the web interface.
- **Web-Based Monitoring:** Real-time video streams, zone status, and fan operation data are accessible remotely.
- **Multi-Zone Support:** The system can be expanded to manage multiple zones within a building.
- **User-Friendly Interface:** Accessible through mobile or desktop platforms for easy management.

Chapter 2

Literature Survey

2.1 Literature Survey

Sl. No	Title	Author & Year	Methodology	Metrics	Advantages	Disadvantages
1	YOLOv3: An Incremental Improvement	J. Redmon, A. Farhadi, 2018	YOLOv3 deep learning-based real-time object detection	Detection accuracy, Speed	High-speed detection, suitable for real-time applications	Requires powerful GPU for optimal performance
2	Realtime Multi-Person 2D Pose Estimation Using Part Affinity Fields	Z. Cao et al., 2017	Pose estimation using Part Affinity Fields (PAF) and deep learning	Pose accuracy, Processing time	Enables multi-person tracking, effective in dynamic environments	Computationally expensive
3	Enhanced Real Human Detection and People Counting using YOLOv8	Tahreer Abdul Ridha Shyaa, Ahmed A Hashim 2024	Utilized YOLOv8 for real-time human detection and counting in images, videos	MAP close to 100% Real – time processing capabilities	High accuracy in crowded and complex scenarios.	Potential challenges in adapting to extreme environmental conditions.
4	YOLOv8: Real-Time Object Detection	Ultralytics, 2023	YOLOv8 deep learning model for object detection and tracking	Detection accuracy, Speed	Improved accuracy and efficiency compared to previous YOLO versions	Requires model fine-tuning for specific use cases
5	Flask: Lightweight Web Framework	Flask Documentation	Micro web framework for Python applications	Response time, Scalability	Easy to integrate with machine learning models and APIs	Limited built-in features, requires extensions for advanced functionality
6	ESP32: IoT and Edge Computing Platform	Espressif, 2023	Low-power microcontroller with Wi-Fi and Bluetooth for IoT applications	Power efficiency, Processing speed	Supports real-time processing, energy-efficient, and cost-effective	Limited computational power for complex ML models

Chapter 3

Proposed System

3.1 Objective

The primary objective of the Smart Motion and Zone-Based Fan Control System Using YOLO is to design an intelligent and energy-efficient fan control system that uses real-time object detection to detect human presence in specific zones and control fans accordingly.

The system aims to:

- Enhance Energy Efficiency by ensuring fans operate only in occupied zones.
- Provide Real-Time Detection using YOLO for accurate motion detection.
- Offer Web-Based Control for monitoring and manually overriding fan operations.
- Ensure User Comfort by automating fan control based on occupancy.

3.2 Methodology

The system follows a step-by-step methodology:

1. Object Detection using YOLO
 - YOLO processes live camera feeds to detect humans and map their positions to predefined zones.
2. Zone-Based Decision Making
 - The system checks which zone is occupied and determines whether the corresponding fan should be switched on or off.
3. Microcontroller Control
 - The microcontroller (ESP32 or Arduino) receives zone data and controls fan operations.
4. Web Interface
 - A web-based dashboard built using HTML, CSS, and JavaScript allows users to monitor zones, view live video feeds, and control fans manually if required.
5. Energy Efficiency Implementation
 - If no motion is detected for 5 seconds, the fan is turned off to conserve energy.

3.2.1 System Architecture

The system architecture consists of the following components:

- Camera: Captures real-time video feed.
- YOLOv8 Model: Processes the video feed to detect people.
- Microcontroller: Controls fan operations based on the detection results.

- Web Server: Handles user requests for live video streams and manual fan control.
- User Interface: Displays video feed, zone status, and fan control options.

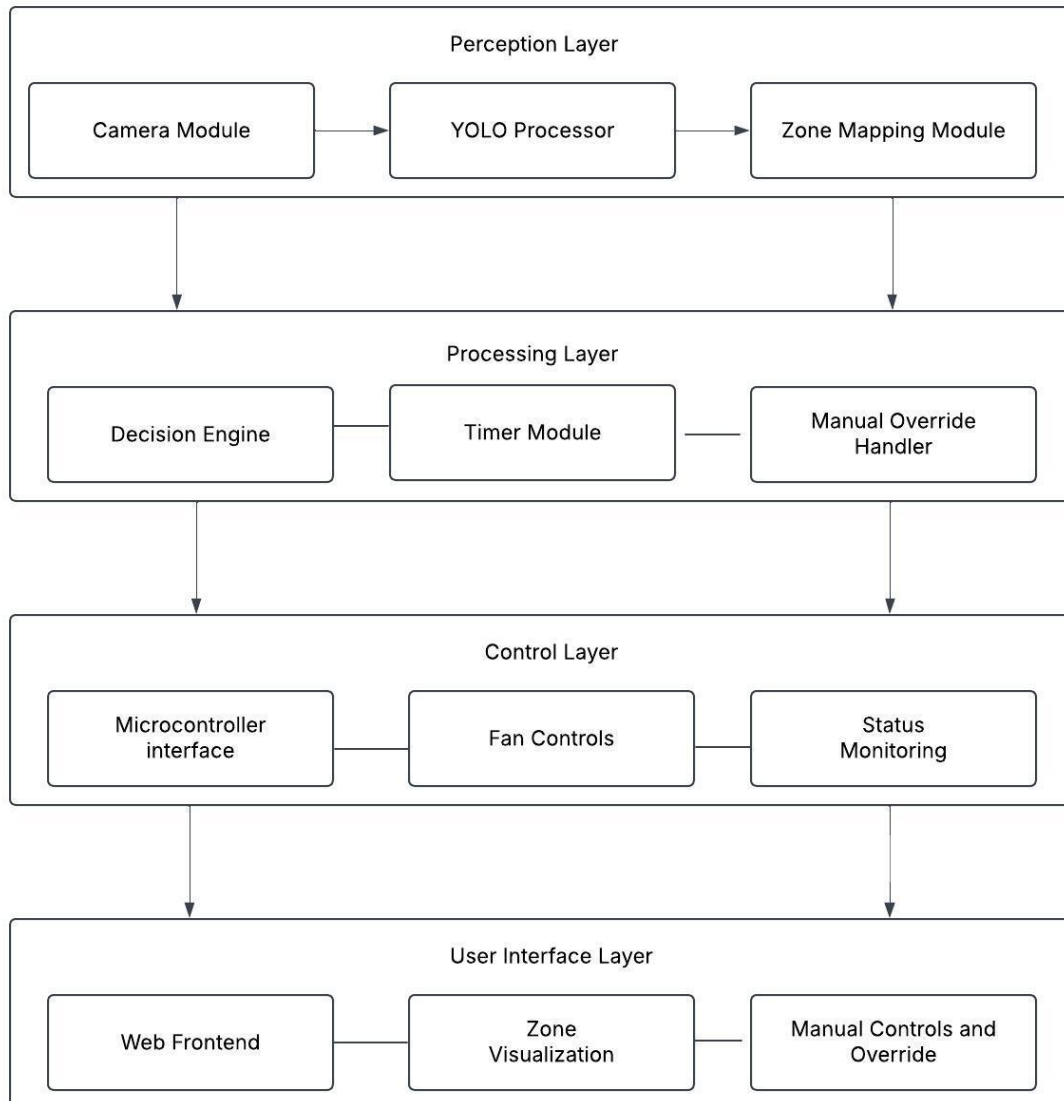


Figure 3.1 System Architecture

3.2.2 Functional Requirements

- Detect human presence using YOLO.
- Map detected persons to specific zones.
- Turn on fans when a zone is occupied.
- Turn off fans when a zone is unoccupied for 5 seconds.
- Provide real-time video feed on the web interface.
- Allow manual fan control using the interface.

3.2.3 Non-Functional Requirements

- Real-time processing with minimal latency.
- Secure communication between the web interface, server, and microcontroller.
- User-friendly interface accessible from both mobile and desktop.
- Energy-efficient fan operation.

3.2.4 Hardware Requirements

- Camera: For capturing live video feed.
- ESP32 / Arduino: For fan control.
- Fans: Controlled via relays.
- Server: For hosting the web interface.

3.2.5 Software Requirements

- Python: For running YOLO object detection.
- Flask: Backend for web interface and data handling.
- OpenCV: For video feed processing.
- HTML, CSS, JavaScript: For building the frontend.

3.2.6 Life Cycle Model

The Iterative Development Model is used for this project. The development involves continuous testing, evaluation, and refinement of the system. Each cycle consists of:

1. Planning: Define objectives and requirements.
2. Design: Develop the architecture and flow diagrams.
3. Implementation: Code the YOLO detection, microcontroller communication, and web interface.
4. Testing: Perform unit testing and system testing.
5. Evaluation: Evaluate performance and refine the system.

3.2.7 Feasibility Study

- Technical Feasibility:

The project uses YOLOv8 for real-time detection, which is efficient and supports multi-object detection. The microcontroller setup ensures accurate control of fans.

- **Economic Feasibility:**
The system is cost-effective, using affordable components like ESP32, Arduino, and webcams.
- **Operational Feasibility:**
The web interface ensures easy operation and monitoring. Users can manually control fans if needed.
- **Schedule Feasibility:**
The development timeline is manageable with iterative implementation, testing, and refinement.

Chapter 4

System Design

4.1 Use Case Diagram

The **Use Case Diagram** represents the interaction between the users and the system. It includes actions like monitoring zones, controlling fans, and switching between automatic and manual modes.

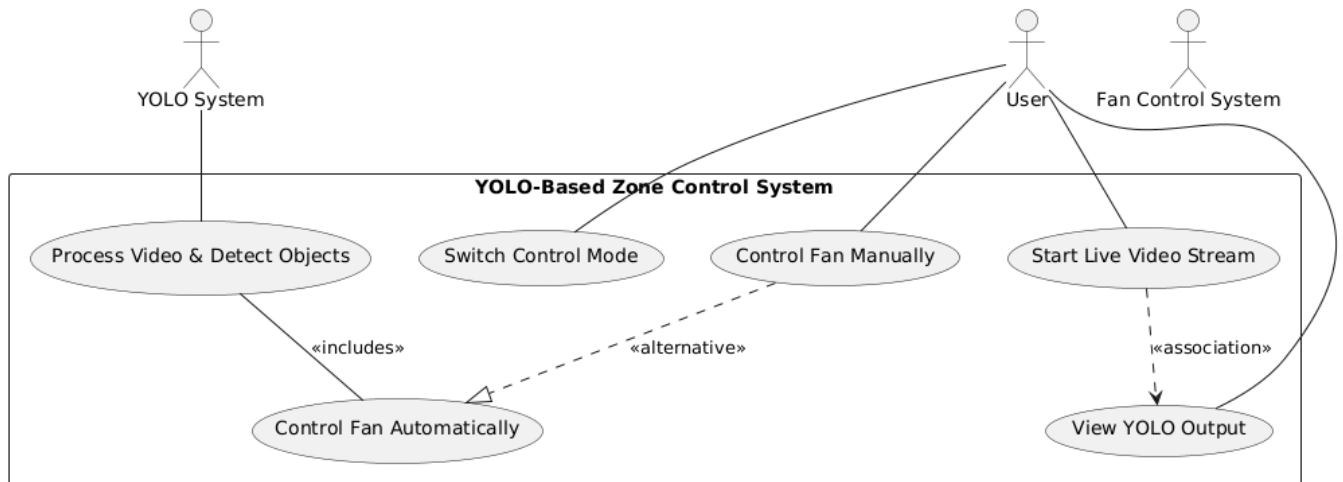


Figure 4.1 Use Case Diagram

Actors:

- **YOLO System:** Detects human presence using YOLOv8 and sends data to control the fans.
- **User:** Interacts with the system to switch modes, manually control fans, and view YOLO output.
- **Fan Control System:** Controls the fans based on the mode and YOLO detection data.

Use Cases:

- **Process Video & Detect Objects:** The YOLO system processes live video streams to detect objects.
- **Control Fan Automatically:** In **Auto Mode**, the fan is controlled based on human detection.
- **Control Fan Manually:** In **Manual Mode**, the user has direct control over the fan operations.
- **Switch Control Mode:** The user can switch between **Auto Mode** and **Manual Mode** using the interface.
- **Start Live Video Stream:** The system provides a live video stream for real-time monitoring.
- **View YOLO Output:** Users can view the detection results to monitor zone occupancy.

Relationships:

- **Includes:** The **Control Fan Automatically** use case is included in the object detection process for automated control.
- **Alternative:** **Control Fan Manually** is an alternative to automatic control when the user opts for manual operation.
- **Association:** The **Start Live Video Stream** and **View YOLO Output** are associated, providing users with detection insights.

4.2 Class Diagram

The **Class Diagram** showcases the structure of the system by representing the classes, attributes, and relationships among objects. The main classes include:

- **Camera**: Captures real-time video feeds.
- **YOLO Processor**: Processes video to detect humans.
- **Zone Manager**: Manages zone-wise detection and control.
- **Fan Controller**: Controls the fans based on detection data.
- **User**: Accesses the web interface for monitoring and control.

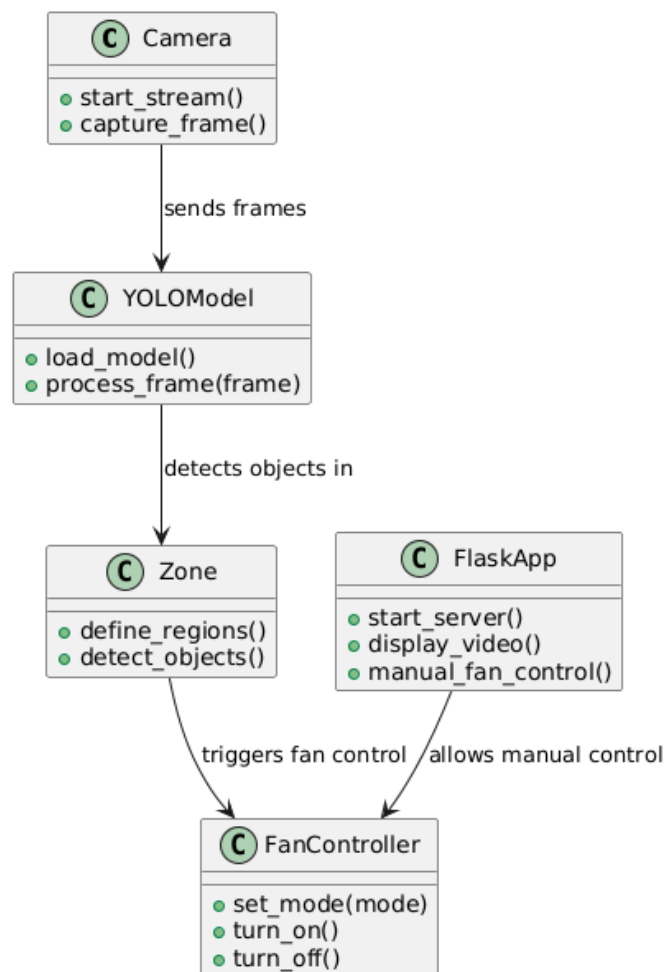


Figure 4.2 Class Diagram

4.3 Activity Diagram

The **Activity Diagram** illustrates the interaction between system components during fan operation. The primary flow is as follows:

1. Camera captures live video.
2. YOLO detects human presence.
3. Detection data is sent to the Zone Manager.
4. Zone Manager updates the Fan Controller.
5. Fans are turned on or off.
6. The web interface displays the real-time status.

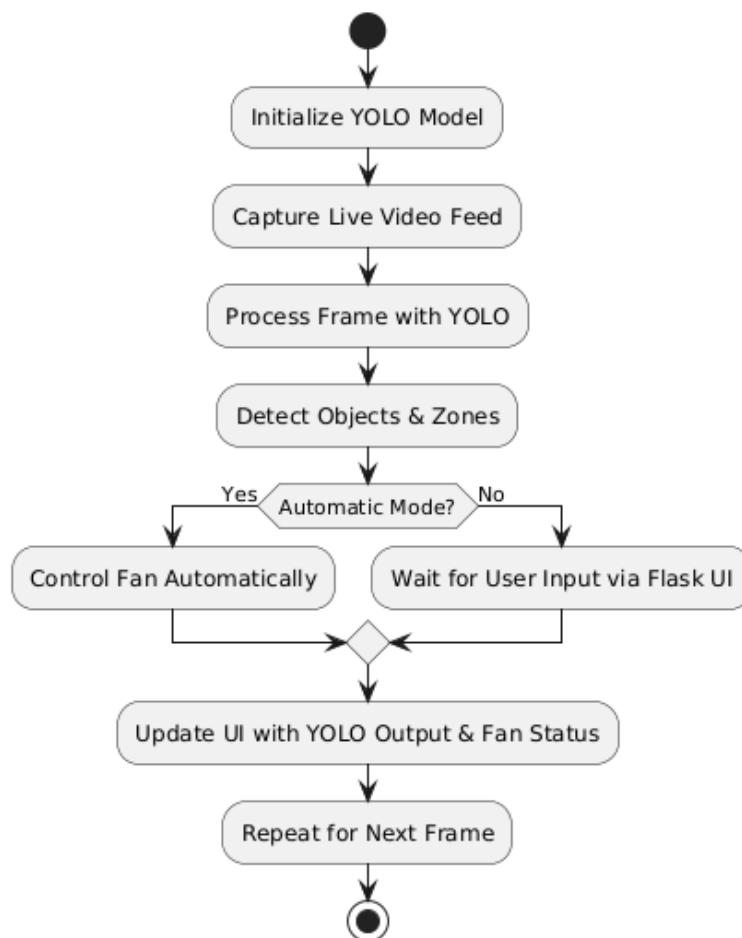


Figure 4.3 Activity Diagram

4.4 Deployment Diagram

The **Deployment Diagram** provides an overview of how different hardware and software components are deployed and connected:

- **Camera** captures video and sends data to the system.
- **YOLO Processor** runs on a computer for object detection.
- **Microcontroller (ESP32)** controls the fan using relay modules.
- **Web Server** handles user interaction via the web interface.

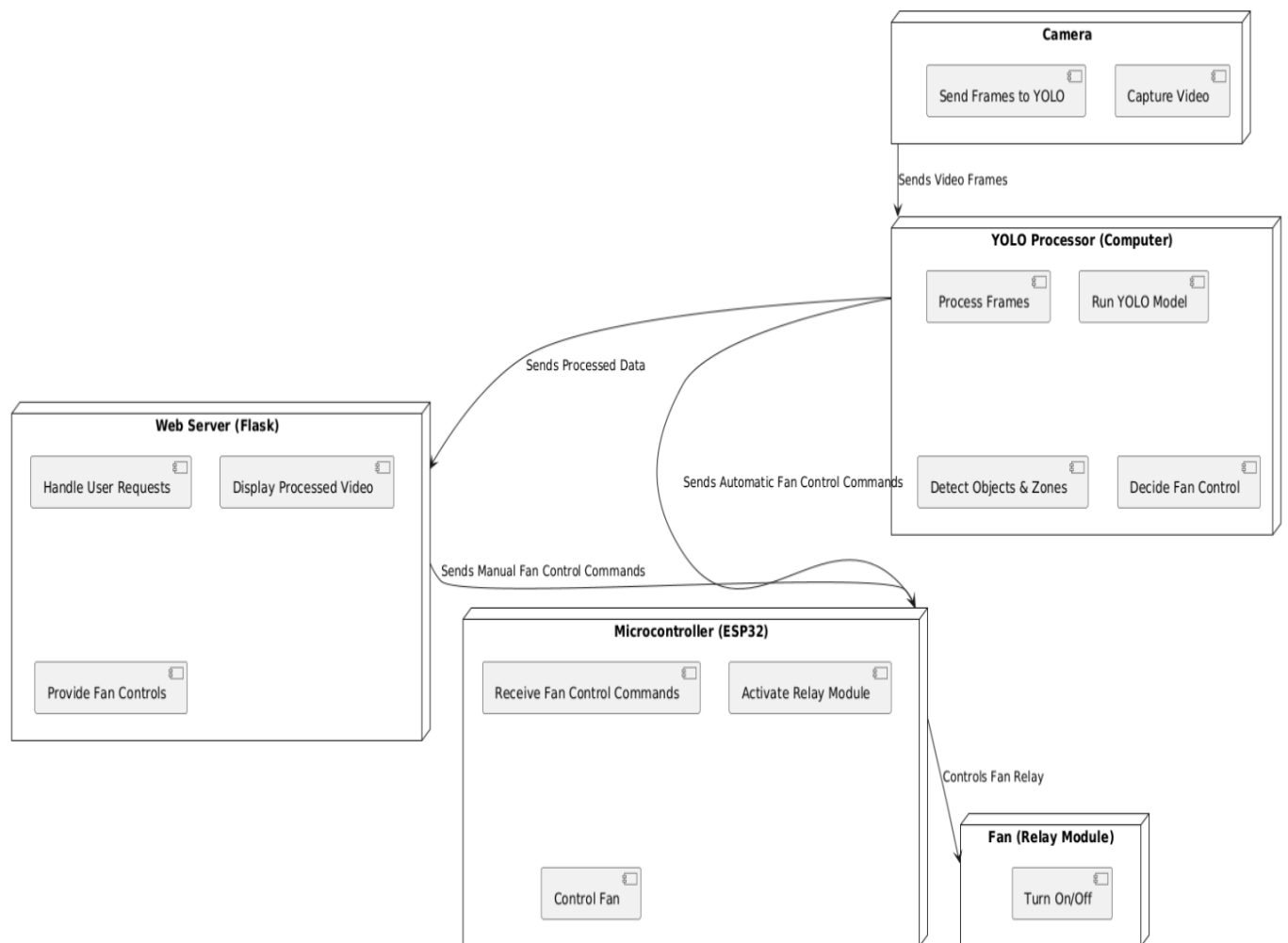


Figure 4.4 Deployment Diagram

4.5 GUI Design

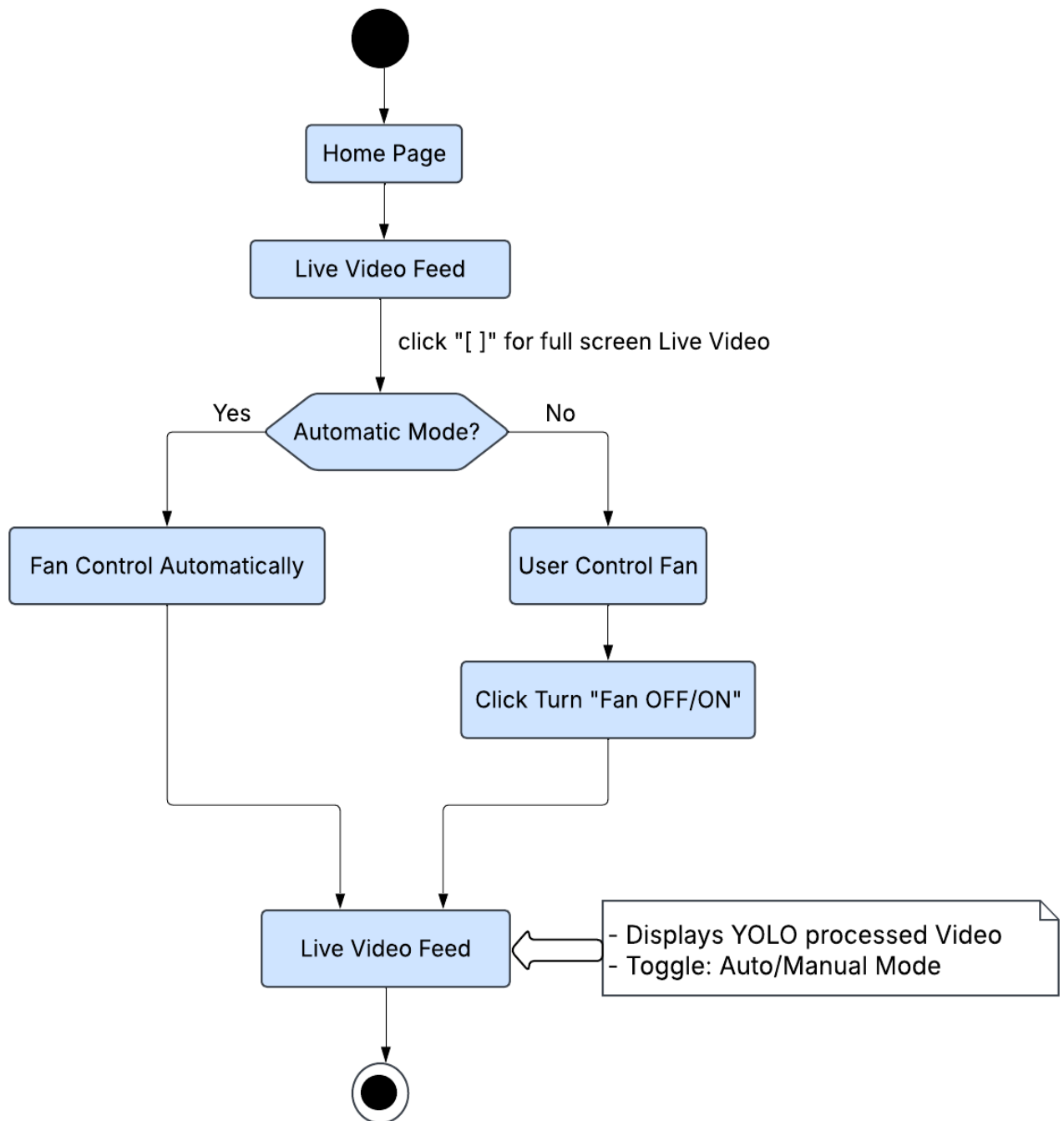


Figure 4.5 GUI Diagram

Chapter 5

Architecture

5.1 Flow Chart

The **Flow Chart** represents the step-by-step process followed by the system to detect human presence, manage zone states, and control fans.

Flow of the System

1. **Video Capture:** The camera captures real-time video.
2. **YOLO Detection:** The YOLO model processes the video frame for object detection.
3. **Zone Mapping:** Detected humans are mapped to their respective zones.
4. **Decision Making:**
 - If a zone is occupied, the corresponding fan is turned on.
 - If a zone is empty for **5 seconds**, the fan is turned off.
5. **Manual Control (If applied):** Users can override the system using the web interface.
6. **Status Update:** The web interface is updated to display the current fan and zone status.

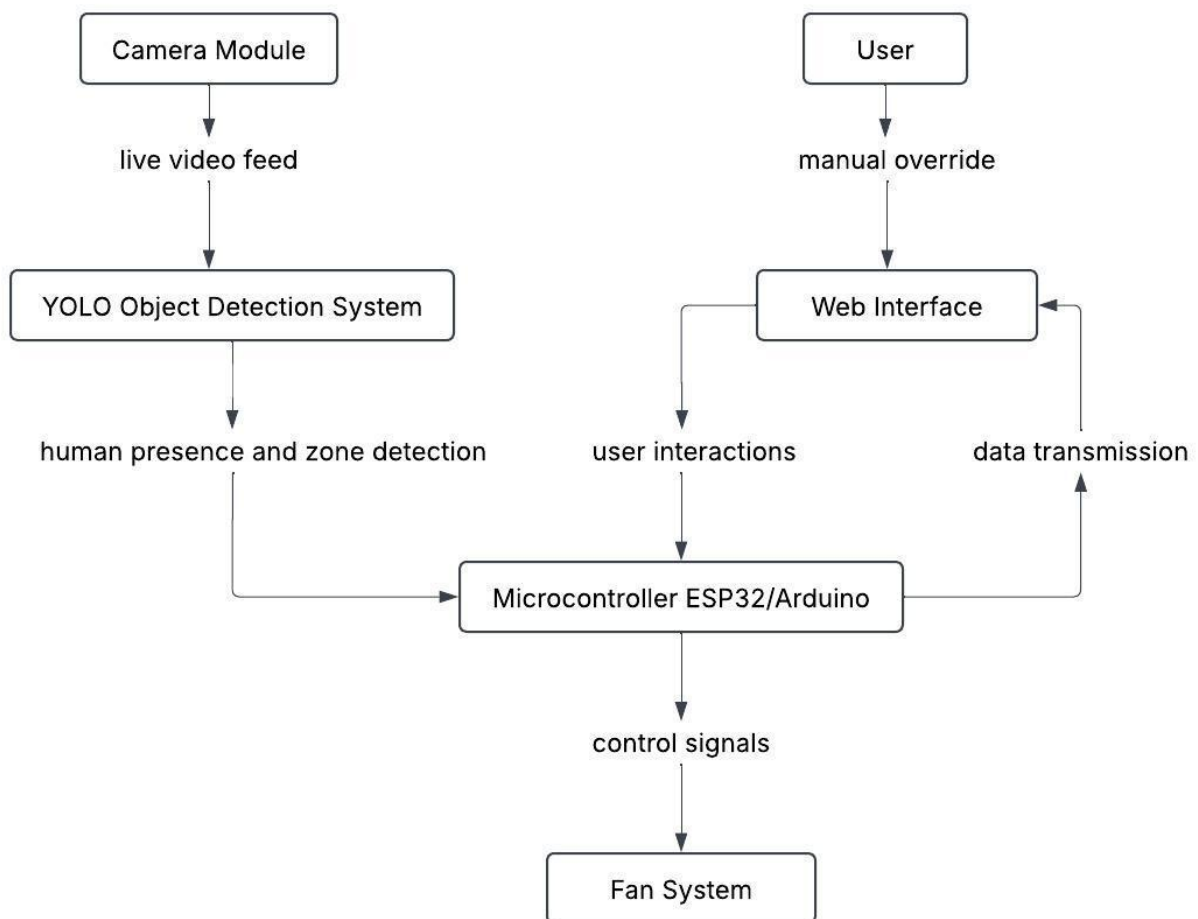


Figure 5.1 Work Flow

5.2 Data Collection

The system collects the following data:

- **Real-Time Video Data:** Captured from the camera for YOLO processing.
- **Detection Data:** Includes the position of detected humans and mapped zones.
- **Fan Status:** Tracks whether fans are **ON** or **OFF** in respective zones.
- **User Input Data:** From the web interface for manual controls.

This data is processed in real-time for immediate fan control and user interface updates.

5.3 Module Descriptions

The system consists of the following modules:

1. Video Capture Module

- Captures live video using a connected camera.
- Sends frames to the YOLO detection module.

2. YOLO Detection Module

- Processes video frames using the **YOLOv8** model.
- Identifies human presence and provides coordinates.

3. Zone Management Module

- Maps detected humans to specific zones.
- Tracks zone occupancy status.

4. Fan Control Module

- Uses a microcontroller (**ESP32** or **Arduino**) to control fan operations.
- Turns fans **ON** or **OFF** based on zone status.

5. Web Interface Module

- Provides a user-friendly interface for real-time monitoring.
- Allows users to switch between **Auto Mode** and **Manual Mode**.
- Displays current zone status and fan states.

Chapter 6

Testing

Testing is a crucial phase in the project to ensure that the system works as expected. Various test cases were created to verify the functionality, performance, and reliability of the **Smart Motion and Zone-Based Fan Control System Using YOLO**.

6.1 Test Case of Module - 1: YOLO Detection

Test ID	Test Description	Input	Expected Output	Actual Output	Status
TC1.1	Detect human in the camera frame	Live video feed	Human detected with bounding box	As expected	Pass
TC1.2	Detect multiple humans	Video with multiple people	All humans detected	As expected	Pass
TC1.3	False detection test	No human in frame	No human detected	As expected	Pass

6.2 Test Case of Module - 2: Zone Mapping

Test ID	Test Description	Input	Expected output	Actual output	Status
TC2.1	Correct zone detection	Human in zone 1	Zone 1 marked as active	As expected	Pass
TC2.2	Multiple zone detection	Human in zone 1 and 3	Both zone 1 and 3 active	As expected	Pass
TC2.3	Zone empty after 5 seconds	Human exit zone 2	Fan turns off in zone 2	As expected	Pass

6.3 Test Case of Module - 3: Fan Control

Test ID	Test Description	Input	Expected Output	Actual Output	Status
TC3.1	Auto mode fan control	Human in zone 3	Fan 3 turns ON	As expected	Pass
TC3.2	Auto mode fan OFF	No human in zone 4	Fan 4 turns OFF	As expected	Pass
TC3.3	Manual override	User switches fan 2 ON	Fan 2 remains ON until manually OFF	As expected	Pass

6.4 Test Case of Module - 4: Web Interface

Test ID	Test Description	Input	Expected Output	Actual Output	Status
TC4.1	Live video feed display	User accesses interface	Real – time video displayed	As expected	Pass
TC4.2	Zone status display	Human enter zone 1	Zone 1 marked Active	As expected	Pass
TC4.3	Manual control	User toggles fan 3	Fan 3 turns ON/OFF accordingly	As expected	Pass

6.5 Summary of Testing

- All modules were tested under different scenarios.
- **Edge Cases** such as sudden movement, overlapping zones, and multiple humans were handled efficiently.
- The web interface provided a seamless user experience for both real-time monitoring and manual control.
- The system accurately detected human presence and managed fan operations without delays.

The test results confirm the successful implementation of the system. Any minor bugs identified were fixed, and the final version demonstrated reliable performance.

Chapter 7

Sample Code

This chapter includes sample code for key components of the **Smart Motion and Zone-Based Fan Control System Using YOLO**. The code is divided into the following sections:

- **7.1 Frontend Code**
- **7.2 Backend Code**
- **7.3 Microcontroller Code**

7.1 Frontend Code (HTML, CSS, and JavaScript)

The frontend interface was built using **HTML**, **CSS**, and **JavaScript**. The user can monitor real-time video feeds, check zone statuses, and control fans using a switch-style button.

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Smart Fan Control</title>
  <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
/>
  <style>
    /* ... (keep your existing CSS styles) ... */
    body {
      background-color: #f0f4f8;
      color: #333;
      font-family: Arial, sans-serif;
    }
    .header {
      background: linear-gradient(90deg, #228afa, #00c6ff);
      color: white;
      padding: 15px;
      text-align: center;
      font-size: 24px;
      font-weight: bold;
    }
    .zone-card {
      background: white;
      border-radius: 12px;
      box-shadow: 0 4px 8px rgba(0, 0, 0, 0.1);
      padding: 20px;
      margin-bottom: 20px;
      transition: background-color 0.5s ease, color 0.5s ease, transform 0.3s ease; /* Added transitions
*/
    }
    .zone-card:hover {
      transform: translateY(-5px);
    }
  </style>
```

```
.active-zone { /* Style for when person is detected */
  background-color: #e8f5e9; /* Light green background */
}
.inactive-zone { /* Style for when no person is detected */
  background-color: white;
}
.fan-state-on {
  color: #2abe1a; /* Green text for ON */
  font-weight: bold;
}
.fan-state-off {
  color: #ff4d4d; /* Red text for OFF */
  font-weight: bold;
}
.switch-container span {
  font-weight: bold;
}
.switch-row {
  display: flex;
  align-items: center;
  gap: 10px;
  margin-top: 10px; /* Added margin */
}
.manual-on {
  /* Style for the switch toggle when in manual mode */
}
.auto-on {
  /* Style for the switch toggle when in auto mode */
}
.video-container {
  margin-bottom: 20px;
  text-align: center;
}
.video-box {
  border: 3px solid #1e2022;
  border-radius: 12px;
  overflow: hidden;
  display: inline-block;
  width: 640px; /* Adjust size as needed */
  height: 480px; /* Adjust size as needed */
  position: relative;
  background-color: #333; /* Background while loading */
}
.video-box img {
  width: 100%;
  height: 100%;
  object-fit: cover;
}
.fullscreen-btn {
  position: absolute;
  bottom: 10px;
```

```
right: 10px;
background-color: rgba(0, 0, 0, 0.7);
color: white;
border: none;
padding: 8px 12px;
border-radius: 8px;
cursor: pointer;
z-index: 10; /* Ensure button is above image */
}
.camera-icon {
position: absolute;
top: 50%;
left: 50%;
transform: translate(-50%, -50%);
font-size: 50px;
color: rgba(255, 255, 255, 0.5); /* White icon on dark background */
}
</style>
</head>
<body>
<div class="header">Smart Fan Zone-Based Control System</div>

<div class="container mt-4">
<!-- Video Container Updated to Show Live Feed -->
<div class="video-container">
<div class="video-box" id="videoBox">
<!-- Add a placeholder/loading indicator -->
<div id="loadingIndicator" class="camera-icon">📷 Loading...</div>

<button class="fullscreen-btn" onclick="toggleFullScreen()">⌕</button>
</div>
</div>

<!-- Removed the Show/Hide Zone Status button, always show -->
<div id="zoneStatus">
<div class="row" id="zoneContainer">
<!-- Zone cards will be generated by JavaScript -->
</div>
</div>
</div>

<script>
const videoFeed = document.getElementById('videoFeed');
const loadingIndicator = document.getElementById('loadingIndicator');

function videoLoaded() {
loadingIndicator.style.display = 'none';
videoFeed.style.display = 'block';
console.log("Video stream loaded.");
}
}
```

```
function videoError() {
  loadingIndicator.textContent = '✖ Error Loading Video';
  loadingIndicator.style.display = 'block';
  videoFeed.style.display = 'none';
  console.error("Failed to load video stream.");
  // Optionally try reloading after a delay
  // setTimeout(() => { videoFeed.src = "/video_feed?" + new Date().getTime(); }, 5000);
}

function toggleFullScreen() {
  const videoBox = document.getElementById('videoBox');
  if (!document.fullscreenElement) {
    videoBox.requestFullscreen().catch((err) => {
      console.error(`Error attempting to enable full-screen mode: ${err.message}
(${err.name})`);
      alert(`Could not enter full-screen: ${err.message}`);
    });
  } else {
    if (document.exitFullscreen) {
      document.exitFullscreen();
    }
  }
}

// Generate Zone Card HTML
function createZone(zoneNumber) {
  return `
    <div class='col-md-6 col-lg-3'> <!-- Adjusted grid for better spacing -->
      <div id='zoneCard${zoneNumber}' class='zone-card inactive-zone'>
        <h5>Zone ${zoneNumber}</h5>
        <p>Detection: <span id='detection${zoneNumber}'>Inactive</span></p>
        <p>Fan Status: <span id='fanStatus${zoneNumber}' class='fan-state-
off'>OFF</span></p>

        <!-- Auto/Manual Switch -->
        <div class='switch-row'>
          <span>Auto</span>
          <label class='form-check form-switch'>
            <input class='form-check-input' type='checkbox' id='modeSwitch${zoneNumber}'
onchange='toggleMode(${zoneNumber})'> <!-- Use onchange -->
          </label>
          <span>Manual</span>
        </div>

        <!-- Manual Fan Control (shown only when in manual mode) -->
        <div id='manualOptions${zoneNumber}' style='display: none;'>
          <div class='switch-row'>
            <span>Fan OFF</span>
            <label class='form-check form-switch'>
              <input class='form-check-input' type='checkbox'
id='manualFanSwitch${zoneNumber}' onchange='manualFanControl(${zoneNumber})'> <!-- Use
```

onchange -->

```
        </label>
        <span>Fan ON</span>
    </div>
</div>
</div>
</div>`;
}
```

// Toggle Auto/Manual Mode for a Zone

```
function toggleMode(zoneNumber) {
    const modeSwitch = document.getElementById(`modeSwitch${zoneNumber}`);
    const isManual = modeSwitch.checked; // true if Manual is selected
    const manualOptions = document.getElementById(`manualOptions${zoneNumber}`);

    console.log(`Zone ${zoneNumber}: Toggling mode. Manual selected: ${isManual}`);

    fetch("/toggle_mode", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ zone: zoneNumber, is_manual: isManual }),
    })
    .then(response => response.json())
    .then(data => {
        console.log("Toggle Mode Response:", data);
        if (data.success) {
            manualOptions.style.display = isManual ? 'block' : 'none';
            // No need to toggle classes on the switch itself, Bootstrap handles it
            // Force an update status to reflect potential changes immediately
            updateStatus();
        } else {
            console.error("Failed to toggle mode:", data.error);
            alert(`Error toggling mode for Zone ${zoneNumber}: ${data.error}`);
            // Revert the switch state if the API call failed
            modeSwitch.checked = !isManual;
        }
    })
    .catch(error => {
        console.error('Error during toggleMode fetch:', error);
        alert(`Network error toggling mode for Zone ${zoneNumber}.`);
        modeSwitch.checked = !isManual; // Revert on network error
    });
}
```

// Manual Fan Control for a Zone

```
function manualFanControl(zoneNumber) {
    const manualSwitch = document.getElementById(`manualFanSwitch${zoneNumber}`);
    const fanState = manualSwitch.checked ? 1 : 0; // 1 for ON, 0 for OFF

    console.log(`Zone ${zoneNumber}: Manual fan control. Desired state: ${fanState == 1 ? 'ON' : 'OFF'}`);
}
```

```
fetch("/fan_control", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ zone: zoneNumber, state: fanState }),
})
.then(response => response.json())
.then(data => {
  console.log("Manual Fan Control Response:", data);
  if (!data.success) {
    console.error("Failed to control fan manually:", data.error);
    alert(`Error controlling fan for Zone ${zoneNumber}: ${data.error}`);
    // Revert the switch if the API call failed
    manualSwitch.checked = !manualSwitch.checked;
  }
  // Update UI immediately based on expected state, or wait for next /status update
  // Let's rely on the next status update for consistency
})
.catch(error => {
  console.error('Error during manualFanControl fetch:', error);
  alert(`Network error controlling fan for Zone ${zoneNumber}.`);
  manualSwitch.checked = !manualSwitch.checked; // Revert on network error
});
}

// Fetch and Update Status Periodically
function updateStatus() {
  fetch("/status")
  .then(response => {
    if (!response.ok) {
      throw new Error(`HTTP error! status: ${response.status}`);
    }
    return response.json();
  })
  .then(data => {
    // console.log("Status Update:", data); // Log fetched data for debugging
    for (let i = 1; i <= 4; i++) {
      const zoneKey = `Zone ${i}`;
      const card = document.getElementById(`zoneCard${i}`);
      const detectionEl = document.getElementById(`detection${i}`);
      const fanStatusEl = document.getElementById(`fanStatus${i}`);
      const modeSwitch = document.getElementById(`modeSwitch${i}`);
      const manualOptions = document.getElementById(`manualOptions${i}`);
      const manualFanSwitch = document.getElementById(`manualFanSwitch${i}`);

      if (!card || !detectionEl || !fanStatusEl || !modeSwitch || !manualOptions ||
!manualFanSwitch) {
        console.warn(`Elements for Zone ${i} not found.`);
        continue; // Skip if elements are missing
      }

      // Update Detection Status and Card Background
```

```
const isDetected = data.detected[zoneKey];
detectionEl.textContent = isDetected ? 'Active' : 'Inactive';
if (isDetected) {
  card.classList.add('active-zone');
  card.classList.remove('inactive-zone');
} else {
  card.classList.add('inactive-zone');
  card.classList.remove('active-zone');
}

// Update Fan Status Text and Color
const fanState = data.fans[zoneKey]; // 0 or 1
fanStatusEl.textContent = fanState === 1 ? 'ON' : 'OFF';
if (fanState === 1) {
  fanStatusEl.classList.add('fan-state-on');
  fanStatusEl.classList.remove('fan-state-off');
} else {
  fanStatusEl.classList.add('fan-state-off');
  fanStatusEl.classList.remove('fan-state-on');
}

// Update Mode Switch State (Auto/Manual)
const mode = data.modes[zoneKey]; // 'auto' or 'manual'
modeSwitch.checked = (mode === 'manual'); // Checked if manual

// Show/Hide Manual Options based on mode
manualOptions.style.display = (mode === 'manual') ? 'block' : 'none';

// Update Manual Fan Switch State (reflects actual fan state when in manual mode)
manualFanSwitch.checked = (fanState === 1);

}
})
.catch(error => {
  console.error('Error fetching status:', error);
  // Optionally display an error message on the page
});
}

// Initial Setup
const zoneContainer = document.getElementById('zoneContainer');
for (let i = 1; i <= 4; i++) {
  zoneContainer.innerHTML += createZone(i);
}

// Start periodic status updates
setInterval(updateStatus, 2000); // Update every 2 seconds

// Initial status fetch on page load
document.addEventListener('DOMContentLoaded', updateStatus);
```

```
</script>
</body>
</html>
```

7.2 Backend Code (Python with Flask)

The backend is developed using **Python** and **Flask** to handle video streaming and zone-based control.

backend.py

```
import cv2
import requests
import numpy as np
import time
import threading
import io
from ultralytics import YOLO
from flask import Flask, render_template, Response, request, jsonify

# --- Configuration ---
ESP32_IP = "http://192.168.157.211" # Change this to your ESP32 IP
VIDEO_SOURCE = 0 # 0 for default webcam, or path to video file/stream URL
YOLO_MODEL = "yolov8n.pt"
DETECTION_CLASS_ID = 0 # 0 is typically 'person' in COCO dataset
DETECTION_INTERVAL_SECONDS = 5 # Time duration for ON/OFF logic
FRAME_SKIP = 2 # Process every Nth frame to potentially improve performance

# --- Flask App Setup ---
app = Flask(__name__)

# --- Shared Data & Locks ---
# Use locks for thread-safe access to shared variables
latest_frame = None
frame_lock = threading.Lock()

# Store detection status (person present T/F)
detected_zones_shared = {"Zone 1": False, "Zone 2": False, "Zone 3": False, "Zone 4": False}
detected_zones_lock = threading.Lock()

# Store actual fan status (ON=1 / OFF=0) - reflects ESP state
zone_fan_status_shared = {"Zone 1": 0, "Zone 2": 0, "Zone 3": 0, "Zone 4": 0}
zone_fan_status_lock = threading.Lock()

# Store control mode ('auto' or 'manual') for each zone
zone_control_mode_shared = {"Zone 1": "auto", "Zone 2": "auto", "Zone 3": "auto", "Zone 4": "auto"}
zone_control_mode_lock = threading.Lock()

# Timers for auto mode logic (managed only by the detection thread)
zone_on_timers = {"Zone 1": None, "Zone 2": None, "Zone 3": None, "Zone 4": None}
zone_off_timers = {"Zone 1": None, "Zone 2": None, "Zone 3": None, "Zone 4": None}

# --- YOLO Detection Thread ---
def run_detection():
```

```
global latest_frame, detected_zones_shared, zone_fan_status_shared, zone_control_mode_shared

print("Loading YOLO model...")
try:
    model = YOLO(YOLO_MODEL)
    print("YOLO model loaded successfully.")
except Exception as e:
    print(f"Error loading YOLO model: {e}")
    return

print(f"Attempting to open video source: {VIDEO_SOURCE}")
cap = cv2.VideoCapture(VIDEO_SOURCE)

if not cap.isOpened():
    print(f"Error: Could not access video source: {VIDEO_SOURCE}")
    return

print("Video source opened successfully.")
ret, frame = cap.read()
if not ret or frame is None:
    print("Error: Could not read initial video frame.")
    cap.release()
    return

frame_height, frame_width, _ = frame.shape
mid_x, mid_y = frame_width // 2, frame_height // 2
print(f"Frame dimensions: {frame_width}x{frame_height}, Midpoints: ({mid_x}, {mid_y})")

frame_count = 0

while True:
    ret, frame = cap.read()
    if not ret or frame is None:
        print("Warning: Could not read frame, skipping iteration.")
        time.sleep(0.1) # Avoid busy-looping if reading fails
        # Optionally try reopening the capture device if it fails persistently
        # cap.release()
        # cap = cv2.VideoCapture(VIDEO_SOURCE)
        # if not cap.isOpened(): break # Exit if reopen fails
        continue

    frame_count += 1
    if frame_count % FRAME_SKIP != 0:
        # --- Still update the displayed frame even if skipping detection ---
        # Draw zone lines
        cv2.line(frame, (mid_x, 0), (mid_x, frame_height), (255, 0, 0), 1)
        cv2.line(frame, (0, mid_y), (frame_width, mid_y), (255, 0, 0), 1)
        # Optional: Draw status text on skipped frames too if needed
        with frame_lock:
            latest_frame = frame.copy()
            time.sleep(0.01) # Small sleep
```

```
        continue # Skip detection for this frame

# --- Run Detection ---
try:
    results = model(frame, verbose=False) # verbose=False reduces console output
except Exception as e:
    print(f"Error during YOLO prediction: {e}")
    continue

current_detected_state = {"Zone 1": False, "Zone 2": False, "Zone 3": False, "Zone 4": False}

# Process results
for result in results:
    for box in result.bboxes:
        try:
            x1, y1, x2, y2 = map(int, box.xyxy[0])
            cx, cy = (x1 + x2) // 2, (y1 + y2) // 2
            class_id = int(box.cls[0])
            confidence = float(box.conf[0])

            # --- Check if detected object is the target class (person) ---
            if class_id == DETECTION_CLASS_ID:
                zone = None
                if cx < mid_x and cy < mid_y: zone = "Zone 1"
                elif cx >= mid_x and cy < mid_y: zone = "Zone 2"
                elif cx < mid_x and cy >= mid_y: zone = "Zone 3"
                else: zone = "Zone 4"

                if zone:
                    current_detected_state[zone] = True
                    # Draw bounding box
                    label = f"{model.names[class_id]} ({confidence:.2f})"
                    cv2.rectangle(frame, (x1, y1), (x2, y2), (0, 255, 0), 2)
                    cv2.putText(frame, label, (x1, y1 - 10), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0,
255, 0), 2)

        except Exception as e:
            print(f"Error processing detection box: {e}")
            continue # Skip this box if error occurs

# Update shared detection status
with detected_zones_lock:
    detected_zones_shared.update(current_detected_state)

# --- Auto Mode Logic ---
current_time = time.time()
# Acquire necessary locks *before* the loop
with zone_control_mode_lock, zone_fan_status_lock:
    # Create copies to avoid holding locks during potentially slow HTTP requests
    current_modes = zone_control_mode_shared.copy()
    current_fan_states = zone_fan_status_shared.copy()
```

```
# Process each zone for auto control
for zone in zone_on_timers.keys():
    zone_index = list(zone_on_timers.keys()).index(zone) + 1 # Zone number 1-4

    # Only apply auto logic if the zone is in 'auto' mode
    if current_modes[zone] == "auto":
        is_detected = current_detected_state[zone]
        fan_state = current_fan_states[zone]

        if is_detected:
            if fan_state == 0: # Fan is OFF, should we turn it ON?
                if zone_on_timers[zone] is None:
                    zone_on_timers[zone] = current_time # Start ON timer
                elif current_time - zone_on_timers[zone] >= DETECTION_INTERVAL_SECONDS:
                    # --- Send ON command ---
                    try:
                        print(f"AUTO: Sending {zone} -> ON")
                        response = requests.get(f'{ESP32_IP}/control?zone={zone_index}&status=1',
timeout=2)

                        print(f"AUTO: ESP32 Response ({zone} ON): {response.text}")
                        # Update shared status only AFTER successful command (or assume success)
                        with zone_fan_status_lock:
                            zone_fan_status_shared[zone] = 1
                    except requests.exceptions.RequestException as e:
                        print(f"AUTO: Error sending ON command to ESP32 for {zone}: {e}")
                        zone_on_timers[zone] = None # Reset timer after action
                        zone_off_timers[zone] = None # Clear off timer

            # Reset OFF timer since person is present
            zone_off_timers[zone] = None

        else: # Person NOT detected
            if fan_state == 1: # Fan is ON, should we turn it OFF?
                if zone_off_timers[zone] is None:
                    zone_off_timers[zone] = current_time # Start OFF timer
                elif current_time - zone_off_timers[zone] >= DETECTION_INTERVAL_SECONDS:
                    # --- Send OFF command ---
                    try:
                        print(f"AUTO: Sending {zone} -> OFF")
                        response = requests.get(f'{ESP32_IP}/control?zone={zone_index}&status=0',
timeout=2)

                        print(f"AUTO: ESP32 Response ({zone} OFF): {response.text}")
                        # Update shared status
                        with zone_fan_status_lock:
                            zone_fan_status_shared[zone] = 0
                    except requests.exceptions.RequestException as e:
                        print(f"AUTO: Error sending OFF command to ESP32 for {zone}: {e}")
                        zone_off_timers[zone] = None # Reset timer after action
                        zone_on_timers[zone] = None # Clear on timer

            # Reset ON timer since person is not present
```

```
        zone_on_timers[zone] = None
    else:
        # If zone is in manual mode, ensure timers are reset
        zone_on_timers[zone] = None
        zone_off_timers[zone] = None

    # --- Draw Overlay Information on Frame ---
    # Draw zone lines
    cv2.line(frame, (mid_x, 0), (mid_x, frame_height), (255, 0, 0), 1)
    cv2.line(frame, (0, mid_y), (frame_width, mid_y), (255, 0, 0), 1)

    # Display zone labels and status (using latest shared data)
    with detected_zones_lock, zone_fan_status_lock, zone_control_mode_lock:
        # Get consistent copies of shared data
        current_detected = detected_zones_shared.copy()
        current_fans = zone_fan_status_shared.copy()
        current_modes = zone_control_mode_shared.copy()

    for i, zone in enumerate(current_detected.keys()):
        detected_text = 'Active' if current_detected[zone] else 'Inactive'
        fan_text = 'ON' if current_fans[zone] == 1 else 'OFF'
        mode_text = current_modes[zone].upper()

        # Determine position based on zone number (1-4)
        text_x = 10 if (i % 2 == 0) else mid_x + 10 # Left zones vs Right zones
        text_y = (30 if (i < 2) else mid_y + 30) # Top zones vs Bottom zones

        cv2.putText(frame, f'{zone}: Detect={detected_text}',
                    (text_x, text_y), cv2.FONT_HERSHEY_SIMPLEX, 0.6, (255, 255, 255), 2)
        cv2.putText(frame, f'Fan={fan_text} [{mode_text}]',
                    (text_x, text_y + 20), cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 255), 2)

    # --- Update the latest frame for streaming ---
    with frame_lock:
        latest_frame = frame.copy()

    # Small delay to prevent hogging CPU
    time.sleep(0.01)

    # Release resources if loop exits
    print("Detection loop ending. Releasing video capture.")
    cap.release()

# --- Frame Generation for Video Stream ---
def generate_frames():
    global latest_frame
    while True:
        with frame_lock:
            if latest_frame is None:
                # Send a placeholder or wait if no frame is available yet
                time.sleep(0.1)
```

```
        continue
    # Encode the frame in JPEG format
    (flag, encodedImage) = cv2.imencode(".jpg", latest_frame)
    if not flag:
        continue

    # Yield the output frame in the byte format
    yield (b'--frame\r\n'
           b'Content-Type: image/jpeg\r\n\r\n' + bytearray(encodedImage) + b'\r\n')
    time.sleep(0.05) # Control frame rate of stream

# --- Flask Routes ---
@app.route('/')
def index():
    """Render the main HTML page."""
    return render_template('index.html')

@app.route('/video_feed')
def video_feed():
    """Video streaming route."""
    return Response(generate_frames(),
                    mimetype='multipart/x-mixed-replace; boundary=frame')

@app.route('/status')
def get_status():
    """API endpoint to get current zone status."""
    with detected_zones_lock, zone_fan_status_lock, zone_control_mode_lock:
        status = {
            "detected": detected_zones_shared.copy(),
            "fans": zone_fan_status_shared.copy(),
            "modes": zone_control_mode_shared.copy()
        }
    return jsonify(status)

@app.route('/toggle_mode', methods=['POST'])
def toggle_mode():
    """API endpoint to switch between auto/manual mode for a zone."""
    data = request.get_json()
    zone_num = data.get('zone') # Expecting 1, 2, 3, or 4
    is_manual = data.get('is_manual') # Expecting true (for manual) or false (for auto)

    if zone_num is None or not (1 <= zone_num <= 4) or is_manual is None:
        return jsonify({"success": False, "error": "Invalid input"}), 400

    zone_key = f"Zone {zone_num}"
    new_mode = "manual" if is_manual else "auto"

    with zone_control_mode_lock:
        zone_control_mode_shared[zone_key] = new_mode
        print(f"MODE: Set {zone_key} to {new_mode}")
```

```
# No immediate ESP32 action needed here, just changing the mode
return jsonify({"success": True, "zone": zone_key, "mode": new_mode})

@app.route('/fan_control', methods=['POST'])
def fan_control():
    """API endpoint for manual fan control."""
    data = request.get_json()
    zone_num = data.get('zone') # Expecting 1, 2, 3, or 4
    state = data.get('state') # Expecting 1 (ON) or 0 (OFF)

    if zone_num is None or not (1 <= zone_num <= 4) or state is None or state not in [0, 1]:
        return jsonify({"success": False, "error": "Invalid input"}), 400

    zone_key = f"Zone {zone_num}"

    # Check if the zone is actually in manual mode before proceeding
    with zone_control_mode_lock:
        if zone_control_mode_shared[zone_key] != "manual":
            return jsonify({"success": False, "error": f"{zone_key} is not in manual mode"}), 400

    # Send command directly to ESP32
    try:
        print(f"MANUAL: Sending {zone_key} -> {'ON' if state == 1 else 'OFF'}")
        response = requests.get(f"{ESP32_IP}/control?zone={zone_num}&status={state}", timeout=2)
        print(f"MANUAL: ESP32 Response ({zone_key} {'ON' if state == 1 else 'OFF'}):
{response.text}")
        # Update shared status *after* sending command
        with zone_fan_status_lock:
            zone_fan_status_shared[zone_key] = state
        return jsonify({"success": True, "zone": zone_key, "state": state})
    except requests.exceptions.RequestException as e:
        print(f"MANUAL: Error sending command to ESP32 for {zone_key}: {e}")
        # Optionally revert fan status if command failed? Or assume it might have worked?
        # For simplicity, we don't revert the shared state here.
        return jsonify({"success": False, "error": f"ESP32 communication failed: {e}"}), 500

# --- Main Execution ---
if __name__ == '__main__':
    # Start the YOLO detection thread
    print("Starting YOLO detection thread...")
    detection_thread = threading.Thread(target=run_detection, daemon=True)
    detection_thread.start()

    # Run the Flask app
    print("Starting Flask web server...")
    # Use host='0.0.0.0' to make it accessible on your network
    app.run(host='0.0.0.0', port=5000, debug=False, threaded=True) # threaded=True is important
```

7.3 Microcontroller Code (ESP32)

The ESP32 microcontroller handles fan control using GPIO pins and processes requests from the backend.

Arduino IDE Code:

```
#include <WiFi.h>
#include <WebServer.h> // Web server library

const char* ssid = "Nan";    // Your Wi-Fi SSID
const char* password = "nandhs1811"; // Your Wi-Fi password

WebServer server(80); // Initialize web server on port 80

// Define GPIO pins for the relay module (Adjust based on wiring)
const int relayPins[4] = {26, 27, 32, 33};

// Function to handle zone control
void handleZoneControl() {
    if (server.hasArg("zone") && server.hasArg("status")) {
        int zone = server.arg("zone").toInt();
        int status = server.arg("status").toInt();

        if (zone >= 1 && zone <= 4) {
            digitalWrite(relayPins[zone - 1], status ? LOW : HIGH); // Active LOW logic
            server.send(200, "text/plain", "Zone " + String(zone) + " set to " + (status ? "ON" : "OFF"));
            Serial.println("Zone " + String(zone) + " set to " + (status ? "ON" : "OFF"));
        } else {
            server.send(400, "text/plain", "Invalid Zone");
        }
    } else {
        server.send(400, "text/plain", "Missing Parameters");
    }
}

void setup() {
    Serial.begin(115200);

    // Connect to Wi-Fi
    WiFi.begin(ssid, password);
    Serial.print("Connecting to Wi-Fi");

    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }

    Serial.println("\nWiFi connected!");
    Serial.print("ESP32 IP Address: ");
    Serial.println(WiFi.localIP());
}
```

```
// Set relay pins as OUTPUT and turn them OFF initially (HIGH for active LOW logic)
for (int i = 0; i < 4; i++) {
    pinMode(relayPins[i], OUTPUT);
    digitalWrite(relayPins[i], HIGH); // Ensure all relays are OFF initially
}

// Define the homepage route
server.on("/", []() {
    server.send(200, "text/html", "<h1>ESP32 Web Server is Running!</h1>");
});

// Define the route for zone control
server.on("/control", HTTP_GET, handleZoneControl);

server.begin();
Serial.println("HTTP server started");
}

void loop() {
    server.handleClient();
}
```

Explanation

- **Frontend Code:** Provides an interactive UI for real-time monitoring and fan control.
- **Backend Code:** Captures video, processes it using YOLO, and streams the live feed.
- **ESP32 Code:** Controls fan operations based on zone status using GPIO pins.

Chapter 8

Result and Analysis

This chapter presents the results of the **Smart Motion and Zone-Based Fan Control System Using YOLO**. The system was tested for its ability to detect human presence, manage zone-based control, and provide manual override functionality through the web interface.

8.1 Screenshots

The following screenshots demonstrate the key functionalities of the system:

8.1.1. Home Page

- The web interface displays a live video feed from the camera.
- Users can monitor zone status and control fans manually.

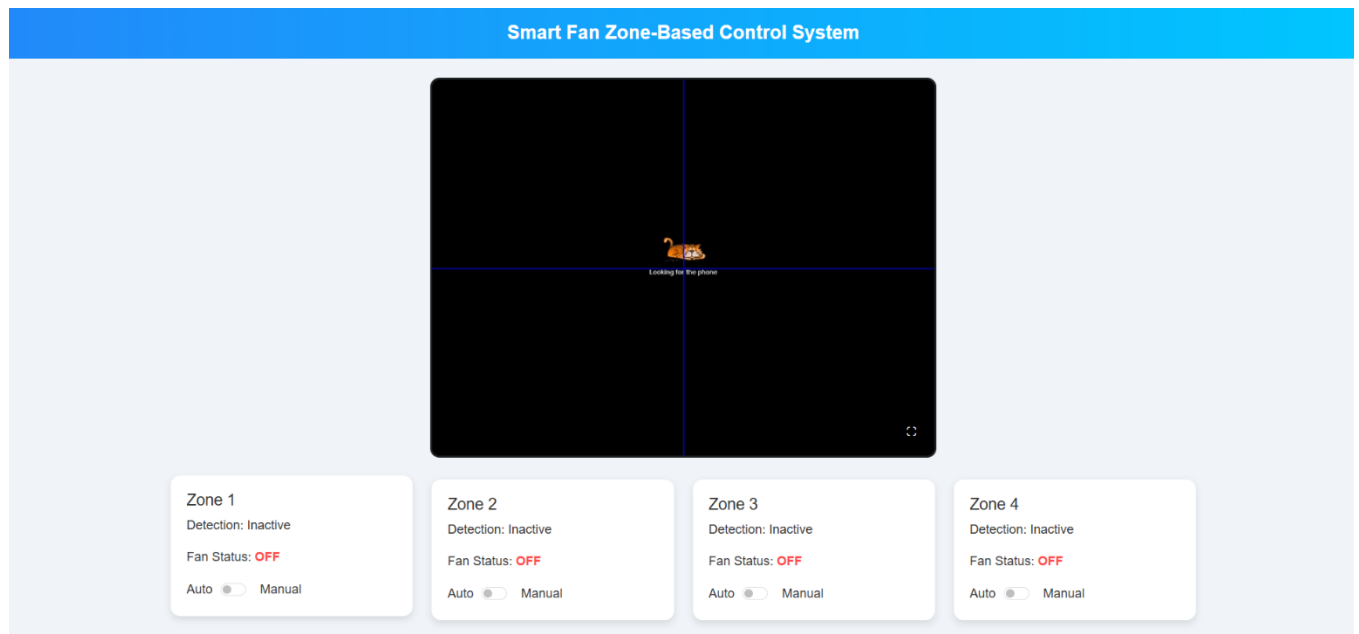
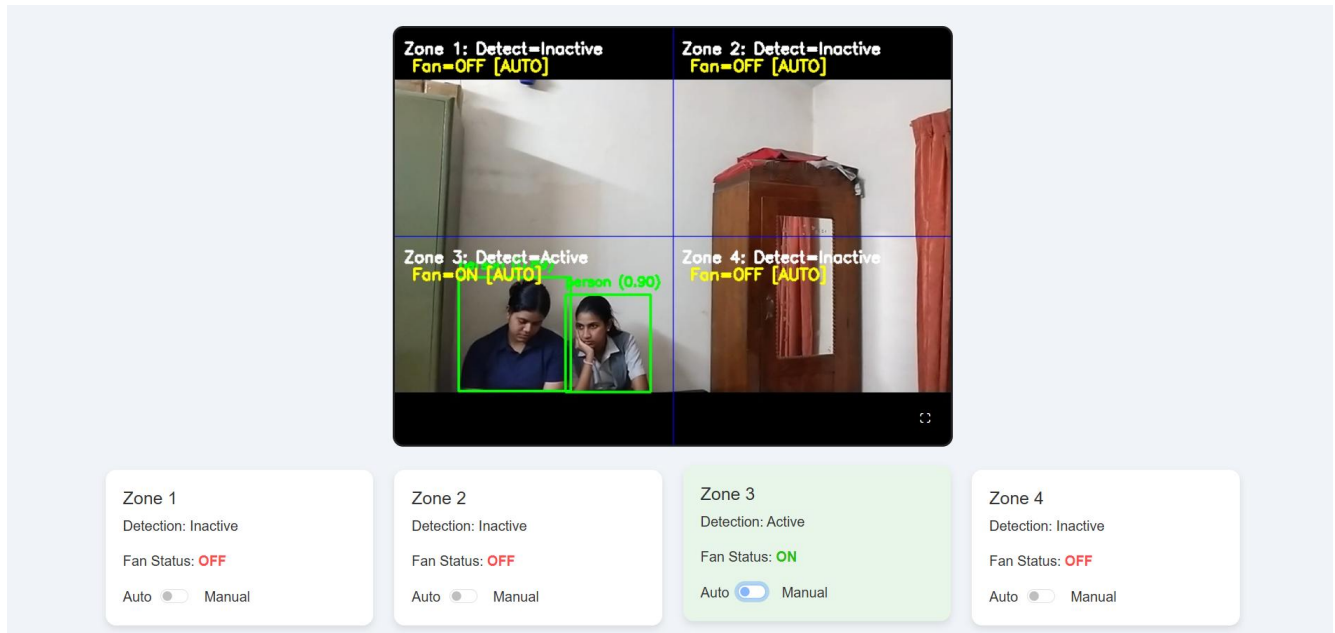


Figure 8.1 Home Page

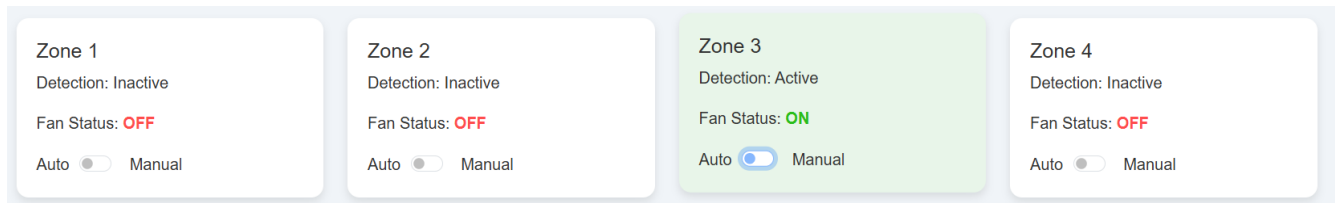
8.1.2. Zone Detection

- The YOLO model successfully detects humans and assigns them to respective zones.
- Zone status updates in real-time on the web interface.



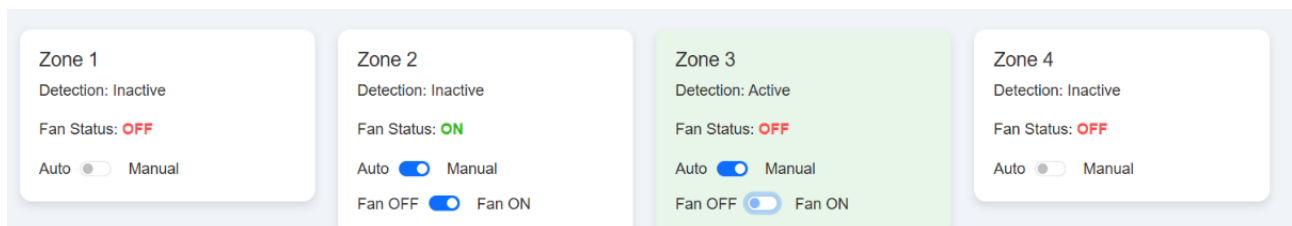
8.1.3. Fan Control (Auto Mode)

- When a human is detected in a specific zone, the fan in that zone is turned **ON** automatically.
- After 5 seconds of inactivity, the fan turns **OFF** to save energy.



8.1.4. Fan Control (Manual Mode)

- Users can manually override the fan status using the switch-style button.
- In manual mode, the switch turns **dark blue** when the fan is on, and **light blue** when it is off.



Chapter 9

Conclusion

The **Smart Motion and Zone-Based Fan Control System Using YOLO** was successfully developed and implemented to provide intelligent fan control based on real-time human detection. By leveraging the YOLOv8 model for accurate object detection and incorporating IoT components like ESP32 for fan control, the system achieved its primary goal of enhancing energy efficiency and user comfort.

Key Achievements:

- Real-time human detection
- Effective zone-based fan control for energy conservation.
- Seamless switching between **Auto Mode** and **Manual Mode**.
- User-friendly web interface for remote monitoring and control.
- Reduced unnecessary energy consumption by turning off fans in unoccupied zones.

Benefits of the System:

- **Energy Efficiency:** Automatically turns off fans in unoccupied zones.
- **User Comfort:** Provides hands-free fan control using advanced object detection.
- **Remote Monitoring:** Users can monitor and control fans from any location.
- **Scalability:** The system can be expanded to cover larger areas with additional zones.

The project's success demonstrates the effectiveness of combining computer vision, IoT, and web technologies for practical energy management. With further improvements, this system could be adapted for larger-scale applications in homes, offices, and commercial spaces.

Chapter 10

Future Enhancement

While the **Smart Motion and Zone-Based Fan Control System Using YOLO** has been successfully implemented, there are opportunities for further enhancements to improve its functionality, scalability, and user experience.

10.1 Enhanced Object Detection

- Integrating **YOLOv8 with Edge AI** using devices like NVIDIA Jetson could reduce latency and enhance real-time performance.
- Implementing **Person Re-Identification (Re-ID)** to track individuals across different zones accurately.

10.2 Advanced Energy Management

- Adding **Adaptive Control Algorithms** that adjust fan speed based on the number of people detected in a zone.
- Implementing predictive analysis using machine learning to optimize fan usage based on occupancy patterns.

10.3 Multi-Zone Expansion

- Supporting larger areas by enabling seamless zone management through additional cameras and microcontrollers.
- Introducing a **Zone Merging Feature** to combine nearby zones dynamically when needed.

10.4 Mobile Application

- Developing a cross-platform **mobile app** for better remote access and notifications.
- Incorporating voice commands using **AI assistants** like Alexa or Google Assistant.

10.5 Enhanced Security

- Implementing **Facial Recognition** to ensure only authorized individuals can control the system.
- Adding **Encrypted Communication** between the camera, microcontroller, and web interface for data privacy.

10.6 Maintenance and Monitoring

- Introducing **Predictive Maintenance Alerts** using AI to detect faults in the fan or microcontroller.
- Providing detailed **Energy Reports** for users to monitor energy savings and fan usage patterns.

These enhancements would further optimize the system's efficiency, provide a richer user experience, and expand its applications to larger spaces and industries.

Chapter 11

References

11.1 References

- [1] J. Redmon and A. Farhadi, "YOLOv3: An Incremental Improvement," arXiv preprint arXiv:1804.02767, 2018.
- [2] Z. Cao, T. Simon, S.-E. Wei, and Y. Sheikh, "Realtime Multi-Person 2D Pose Estimation Using Part Affinity Fields," in **Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)**, 2017, pp. 7291–7299.
- [3] T. A. R. Shyaa and A. A. Hashim, "Enhancing real human detection and people counting using YOLOv8," *BIO Web of Conferences*, vol. 97, p. 00061, Apr. 2024. [Online]. Available: <https://doi.org/10.1051/bioconf/20249700061>.
- [4] A. Graves, S. Fernández, F. Gomez, and J. Schmidhuber, "Connectionist Temporal Classification: Labelling Unsegmented Sequence Data with Recurrent Neural Networks," in **Proceedings of the 23rd International Conference on Machine Learning (ICML)**, 2006, pp. 369–376.